Devless

AI-Powered Code Generation from GitHub Issues

An army of AI workers at your service

The Problem

What I wanted

- Create issues → get working code
- Handle multiple issues in parallel
- Review-ready PRs, not code dumps
- Respond to PR comments, iterate

The dream

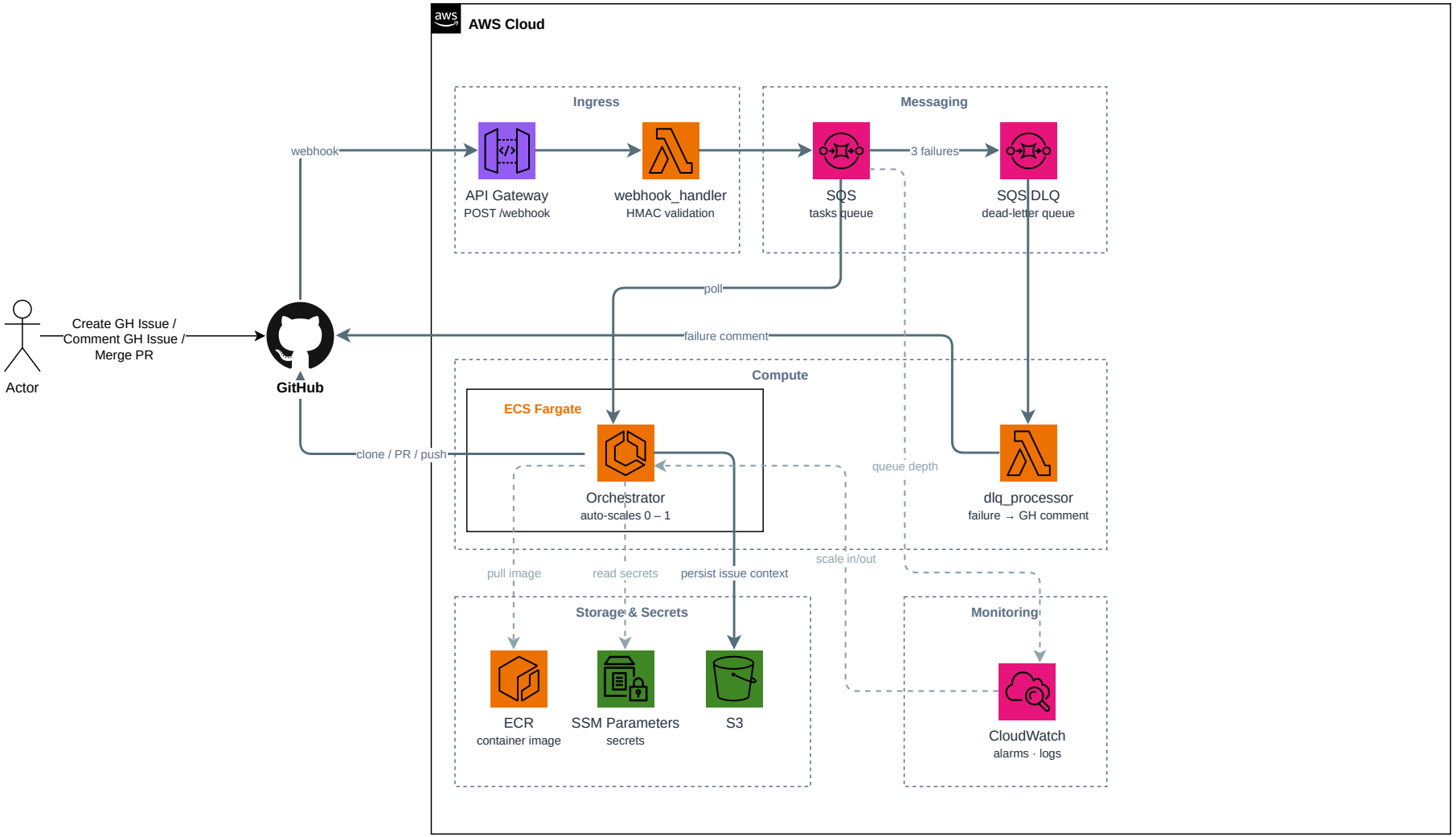
> Open issue → AI implements → PR opens → Review → Merge

Available Tools

Tool	Status	Notes
GitHub Copilot Agent	✅ Active	Zero setup, GitHub-only
Devin	✅ Active	$500/mo
SWE-agent	✅ Open source	DIY setup
Aider	✅ Open source	CLI, no GitHub integration

Gap: No open, affordable, issue-to-PR with AWS/Bedrock.

Solution: Devless



Two Orchestrator Approaches

1. State Machine (Original)

- Python code controls flow
- Explicit states: `pending → cloned → agent_loop → pr_created`
- ~800 lines of code

2. Agent-based (Strands)

- LLM decides flow via SKILL.md files
- Tools for git/GitHub/code operations
- ~400 lines of code

State Machine Approach

```
Dot Executable: /opt/local/bin/dot
Dot executable does not exist
Cannot find Graphviz. You should try

@startuml
testdot
@enduml

 or

java -jar plantuml.jar -testdot
```

Pros: Deterministic, easy to debug

Agent-based Approach

```
Dot Executable: /opt/local/bin/dot
Dot executable does not exist
Cannot find Graphviz. You should try

@startuml
testdot
@enduml

 or

java -jar plantuml.jar -testdot
```

Pros: Flexible, edit SKILL.md for changes

Comparison: Two Approaches

Aspect	State Machine	Agent-based
Control flow	Hardcoded Python	LLM via SKILL.md
Lines of code	~800	~400
Adding workflows	Code changes	Edit SKILL.md
Error handling	try/except	Agent decides
Predictability	✅ Deterministic	⚠️ May vary
Flexibility	⚠️ Rigid	✅ Adapts
Secrets	Hidden	Hidden in tools

Security: Secrets Handling

Both approaches hide secrets from LLM

State Machine: Orchestrator holds token, never passes to Aider

Agent-based: Tools initialized with token at startup

- LLM calls `create_pr(title, body)`
- Tool internally uses token
- Token never in LLM context

vs GitHub Copilot Agent

Aspect	Devless	Copilot Agent
Setup	Terraform deploy	✅ Zero
Cost	~$20/mo	GitHub sub
LLM choice	Bedrock, OpenAI	GitHub only
Customization	Full control	Limited
Self-hosted	✅ Yes	❌ No
Data residency	Your AWS	GitHub
Maintenance	You	GitHub

Demo

Repo: github.com/typekpb/devless-demo

1. Open issue describing feature
2. Devless posts status comment
3. PR opens with implementation
4. Review, comment, iterate
5. Merge

Target Repo Setup

`.devless/config.yml`:

```
model: bedrock/eu.anthropic.claude-sonnet-4-20250514-v1:0
test_command: npm test
```

Webhook events: Issues, Issue comments, PR reviews

That's it. No agents to install.

Infrastructure

```
terraform/
├── modules/
│   ├── api_gateway/   # Webhook endpoint
│   ├── lambda/        # HMAC validation
│   ├── sqs/           # Task queue
│   ├── ecs/           # Orchestrator
│   └── iam/           # Roles
```

One `terraform apply` deploys everything.

Cost (~$20/month)

Service	Cost
Lambda	~$0
SQS	~$0
ECS Fargate	~$10
Bedrock	~$10

AWS sandbox = $0

Lessons Learned

What worked ✅

- Aider — battle-tested
- GitHub App auth — no PATs
- Bedrock — zero config with IAM

Key insight

> State machines feel safe but become rigid.
> Agent + tools = less code, same control.

Recommendation: Start with agent-based.

Q&A

Devless: github.com/typekpb/devless

Demo: github.com/typekpb/devless-demo